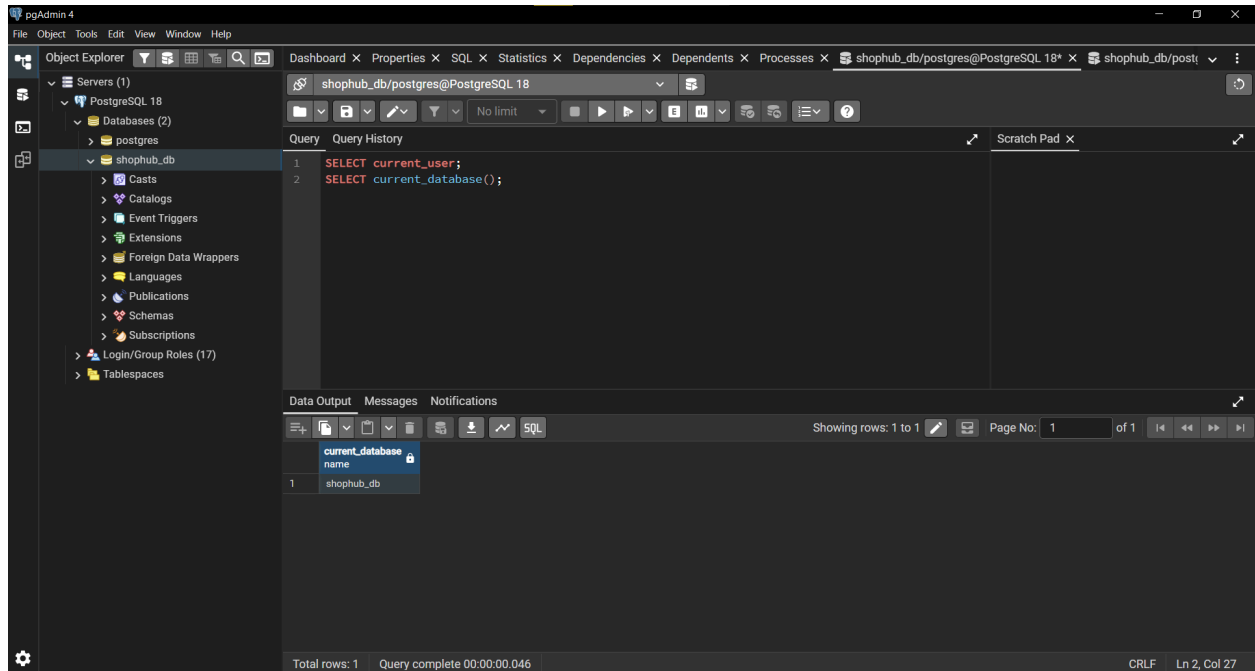
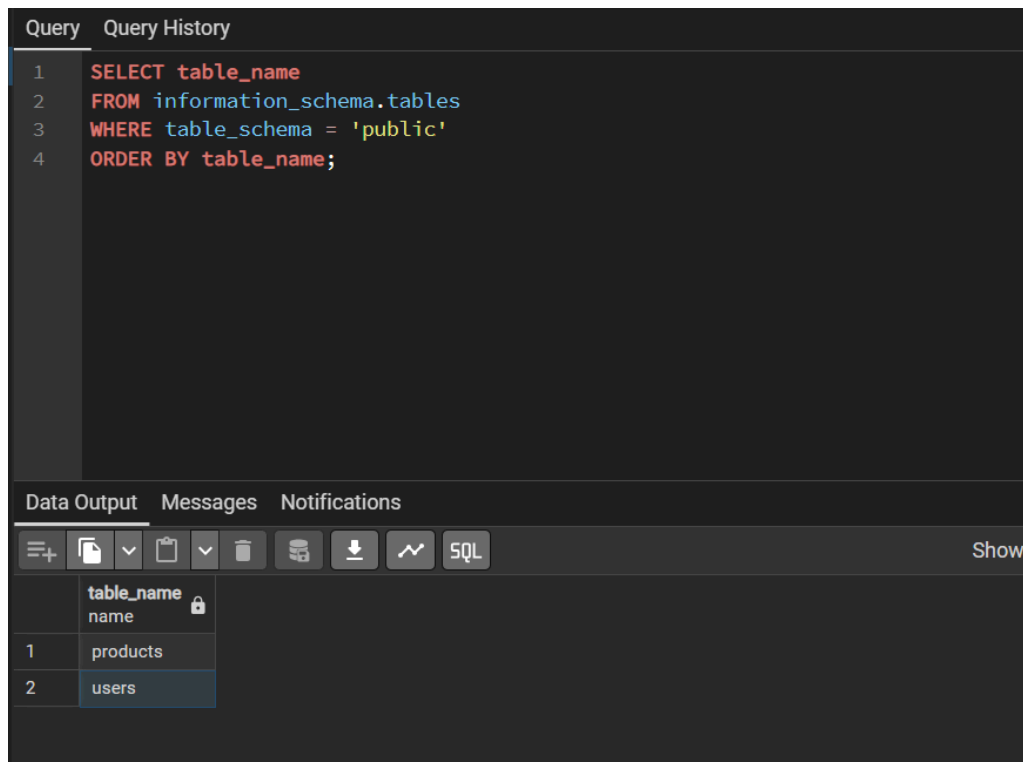


1. PostgreSQL database connection



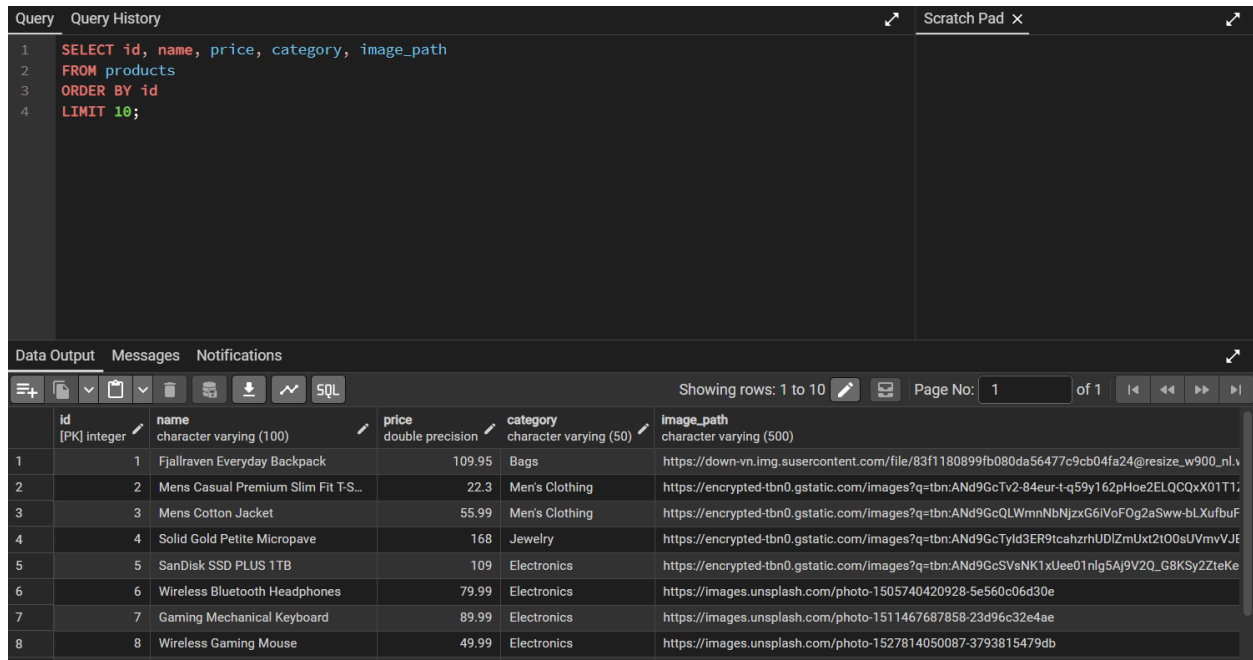
The FastAPI project is using the PostgreSQL database shophub_db with the postgres user.

2. ProductDB and UserDB tables exist



The products and users tables were created in PostgreSQL using SQLAlchemy models.

3. Product data imported/stored in PostgreSQL



The screenshot shows a PostgreSQL query editor with a query window and a results window. The query window contains the following SQL query:

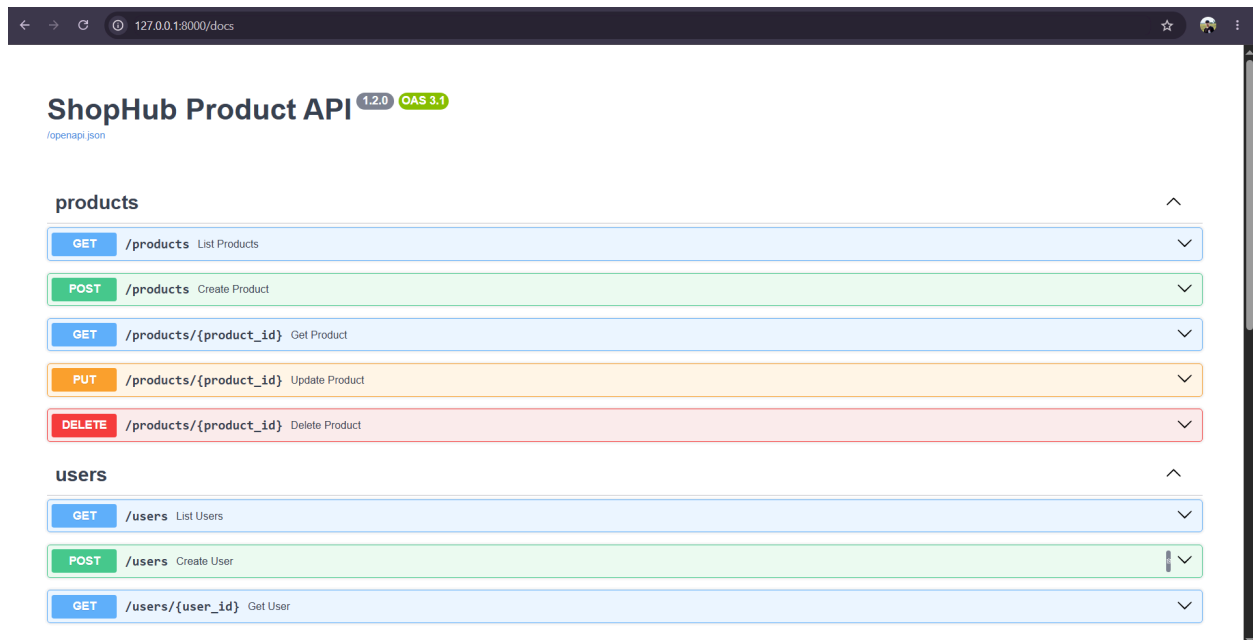
```
1 SELECT id, name, price, category, image_path
2 FROM products
3 ORDER BY id
4 LIMIT 10;
```

The results window displays the output of the query as a table with 8 rows and 5 columns. The columns are: id (integer), name (character varying), price (double precision), category (character varying), and image_path (character varying). The data is as follows:

id	name	price	category	image_path
1	Fjallraven Everyday Backpack	109.95	Bags	https://down-vn.img.susercontent.com/file/83f1180899fb080da56477c9cb04fa24@resize_w900_n1.v
2	Mens Casual Premium Slim Fit T-S...	22.3	Men's Clothing	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTv2-84eur-t-q59y162pHoe2ELQCQxX01T1;
3	Mens Cotton Jacket	55.99	Men's Clothing	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQLWmnNbNjzxG6IVoF0g2aSww-bLXufbuF
4	Solid Gold Petite Micropave	168	Jewelry	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTyld3ER9tcahzrhUDlZmUxt2t00sUvmvVJf
5	SanDisk SSD PLUS 1TB	109	Electronics	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSVsNK1xUee01nlG5Aj9V2Q_G8KSy2ZteKe
6	Wireless Bluetooth Headphones	79.99	Electronics	https://images.unsplash.com/photo-1505740420928-5e560c06d30e
7	Gaming Mechanical Keyboard	89.99	Electronics	https://images.unsplash.com/photo-1511467687858-23d96c32e4ae
8	Wireless Gaming Mouse	49.99	Electronics	https://images.unsplash.com/photo-1527814050087-3793815479db

Existing product data is now stored in the PostgreSQL products table instead of only in products.json.

4. Swagger shows Product CRUD endpoints



The screenshot shows the Swagger UI for the ShopHub Product API. The API is titled "ShopHub Product API" with versions "1.2.0" and "OAS 3.1". The API is defined in the file "openapi.json". The API has two main sections: "products" and "users".

The "products" section contains the following endpoints:

- GET /products: List Products
- POST /products: Create Product
- GET /products/{product_id}: Get Product
- PUT /products/{product_id}: Update Product
- DELETE /products/{product_id}: Delete Product

The "users" section contains the following endpoints:

- GET /users: List Users
- POST /users: Create User
- GET /users/{user_id}: Get User

Swagger UI shows the Product CRUD endpoints implemented for the PostgreSQL backend.

5. GET /products reads from PostgreSQL

Request URL

http://127.0.0.1:8080/products?page=1&size=10

Server response

Code

Details

200

Response body

```
{
  "total": 20,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.99,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://down-vn.img.susercontent.com/file/83f1180899fb080da56477c9cb04fa24@resize_v900_n1.webp"
    },
    {
      "id": 2,
      "name": "Mens Casual Premium Slim Fit T-Shirt",
      "price": 22.9,
      "category": "Men's Clothing",
      "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
      "imageUrl": "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTv2-84eur-t-q59y162pHoe2ELQCxX01T1ZLYkyqsg&s=10"
    },
    {
      "id": 3,
      "name": "Mens Cotton Jacket",
      "price": 55.99,
      "category": "Men's Clothing",
      "description": "Lightweight outerwear jacket suitable for spring, autumn, hiking, camping, and casual daily wear."
    }
  ]
}
```

Response headers

```
content-length: 2904
content-type: application/json
date: Mon, 29 Jun 2026 08:43:11 GMT
server: unicorn
```

-H 'accept: application/json'

Request URL

http://127.0.0.1:8080/products?page=1&size=10

Server response

Code

Details

200

Response body

```
{
  "category": "Electronics",
  "description": "Reliable solid state drive with fast read speeds for upgrading laptops, desktops, and coursework machines.",
  "imageUrl": "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSVsW1xUee0lnlg5Aj3V2Q_6BK5y2tzeke-gHBdn98Kw&s=10"
},
{
  "id": 6,
  "name": "Wireless Bluetooth Headphones",
  "price": 79.99,
  "category": "Electronics",
  "description": "Over-ear wireless headphones with noise isolation, deep bass, and up to 30 hours of battery life.",
  "imageUrl": "https://images.unsplash.com/photo-1505740420928-5e566c66d38e"
},
{
  "id": 7,
  "name": "Gaming Mechanical Keyboard",
  "price": 89.99,
  "category": "Electronics",
  "description": "RGB mechanical keyboard with tactile switches, durable aluminum frame, and customizable lighting.",
  "imageUrl": "https://images.unsplash.com/photo-1511467687858-23d96c32e4ae"
},
{
  "id": 8,
  "name": "Wireless Gaming Mouse",
  "price": 49.99,
  "category": "Electronics",
  "description": "High-precision wireless gaming mouse with adjustable DPI and ergonomic design.",
  "imageUrl": "https://images.unsplash.com/photo-1527814050087-3793815479db"
}
]
```

Response headers

```
content-length: 2904
content-type: application/json
date: Mon, 29 Jun 2026 08:43:11 GMT
server: unicorn
```

GET /products retrieves paginated product data from PostgreSQL.

6. POST /products creates data in PostgreSQL

Request URL
`http://127.0.0.1:8080/products`

Server response

Code	Details
201	Response body <pre>{ "id": 23, "name": "PostgreSQL Test Product", "price": 88.88, "category": "Database Test", "description": "This product was created to test PostgreSQL CRUD.", "imageUrl": "https://via.placeholder.com/380" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 194 content-type: application/json date: Mon, 29 Jun 2026 08:47:20 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Media type:

Controls Accept header:

Example Value | Schema

shophub_db/postgres@PostgreSQL 18

Query

```
1 SELECT id, name, price, category, image_path
2 FROM products
3 WHERE name = 'PostgreSQL Test Product';
```

Scratch Pad

Data Output

	id	name	price	category	image_path
	[PK] integer	character varying (100)	double precision	character varying (50)	character varying (500)
1	23	PostgreSQL Test Produ...	88.88	Database Test	https://via.placeholder.com/3...

Showing rows: 1 to 1 | Page No: 1 of 1

The product created from Swagger is persisted in the PostgreSQL products table.

7. PUT /products/{id} updates PostgreSQL data

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/products/23' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=PostgreSQL Updated Product' \
  -F 'price=99.99' \
  -F 'category=Updated DB Test' \
  -F 'description=This product was updated using PUT and saved in PostgreSQL.' \
  -F 'imageUrl=https://via.placeholder.com/500' \
  -F 'image_file'
```

Request URL

http://127.0.0.1:8000/products/23

Server response

Code Details

200

Response body

```
{
  "id": 23,
  "name": "PostgreSQL Updated Product",
  "price": 99.99,
  "category": "Updated DB Test",
  "description": "This product was updated using PUT and saved in PostgreSQL.",
  "imageUrl": "https://via.placeholder.com/500"
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 289
content-type: application/json
date: Mon, 29 Jun 2026 08:51:47 GMT
server: uvicorn
```

Responses

Code Description Links

PUT /products/{id} updates product information in PostgreSQL.

shophub_db/postgres@PostgreSQL 18

Query

```
1 SELECT id, name, price, category, image_path
2 FROM products
3 WHERE id = 23;
```

Query History

Scratch Pad

Data Output

Showing rows: 1 to 1

Page No: 1 of 1

	id [PK] integer	name character varying (100)	price double precision	category character varying (50)	image_path character varying (500)
1	23	PostgreSQL Updated Prod...	99.99	Updated DB Test	https://via.placeholder.com/5...

The updated product values are saved in the PostgreSQL database.

8. DELETE /products/{id}

The screenshot shows a REST client interface with the following details:

- Curl:** `curl -X 'DELETE' \ 'http://127.0.0.1:8000/products/23' \ -H 'accept: */*'`
- Request URL:** `http://127.0.0.1:8000/products/23`
- Server response:** 204
- Response headers:**
 - `access-control-allow-credentials: true`
 - `date: Mon, 29 Jun 2026 08:53:55 GMT`
 - `server: uvicorn`
- Responses:** A table with two entries:

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links
- Media type:** `application/json`
- Example Value:**

```
{  "detail": [    {      "loc": [        "string",        0      ]    }  ]}
```

DELETE /products/{id} removes a product from PostgreSQL.

The screenshot shows a PostgreSQL query editor interface with the following details:

- Database:** `shophub_db/postgres@PostgreSQL 18`
- Query:**

```
1 SELECT id, name
2 FROM products
3 WHERE id = 23;
```
- Data Output:** A table with two columns:

id	name
----	------

The deleted product no longer exists in the PostgreSQL products table.

9. Filter and pagination homework

The screenshot shows the Curl CLI interface with the following details:

- Curl:** `curl -X 'GET' \ 'http://127.0.0.1:8000/products?category=Bags&page=1&size=2' \ -H 'accept: application/json'`
- Request URL:** `http://127.0.0.1:8000/products?category=Bags&page=1&size=2`
- Server response:** Status code 200.
- Response body:** A JSON object representing a paginated list of products. The first item is a backpack.

```
{  "total": 1,  "page": 1,  "size": 2,  "items": [    {      "id": 1,      "name": "Fjallraven Everyday Backpack",      "price": 169.95,      "category": "Bags",      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",      "imageUrl": "https://down-vn.img.susercontent.com/file/83f1188899fb088da56477c9cb04fa24@resize_v_900_n1.webp"    }  ]}
```
- Response headers:**

```
content-length: 344
content-type: application/json
date: Mon, 29 Jun 2026 08:55:39 GMT
server: unicorn
```
- Responses table:**

Code	Description	Links
200	GET /products?category=Bags&page=1&size=2	

GET /products supports SQLAlchemy filtering and pagination using category, page, and size query parameters.

10. User endpoints

The screenshot shows the Swagger UI for the 'users' endpoint group. It lists three endpoints:

- GET /users**: List Users
- POST /users**: Create User
- GET /users/{user_id}**: Get User

Swagger UI shows basic User endpoints for future authentication features.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "testuser@example.com",
    "full_name": "Test User",
    "password": "password123",
    "role": "customer"
  }'
```

Request URL

http://127.0.0.1:8000/users

Server response

Code	Details
201	Response body <pre>{ "id": 2, "email": "testuser@example.com", "full_name": "Test User", "role": "customer" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 81 content-type: application/json date: Mon, 29 Jun 2026 08:59:04 GMT server: uvicorn</pre>

Responses

POST /users creates a user while hiding sensitive password_hash from the public response.

11.GET /users and GET /users/{id}

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/users

Server response

Code	Details
200	Response body <pre>{ [{ "id": 2, "email": "testuser@example.com", "full_name": "Test User", "role": "customer" }] }</pre> Response headers <pre>content-length: 83 content-type: application/json date: Mon, 29 Jun 2026 09:00:34 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

GET /users returns public user data using UserRead and does not expose password_hash.

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/users/2' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/2
```

Server response

Code	Details
200	Response body <pre>{ "id": 2, "email": "testuser@example.com", "full_name": "Test User", "role": "customer" }</pre> Download Response headers <pre>content-length: 81 content-type: application/json date: Mon, 29 Jun 2026 09:01:19 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

GET /users/{id} retrieves safe user details from PostgreSQL.

12. Prove password_hash exists internally but hidden publicly

The screenshot shows a PostgreSQL database client interface. At the top, the connection is set to 'shophub_db/postgres@PostgreSQL 18'. Below the connection bar is a toolbar with various icons for file operations, filters, and execution. The main area is divided into two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query:

```
1 SELECT column_name
2 FROM information_schema.columns
3 WHERE table_name = 'users'
4 ORDER BY ordinal_position;
```

Below the query editor are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table of results. The table has two columns: 'column_name' and 'name'. The results are as follows:

	column_name	name
1	id	
2	email	
3	full_name	
4	password_hash	
5	role	
6	created_at	

At the bottom right of the 'Data Output' tab, it says 'Showing rows: 1 to 6'.

The users table stores password_hash internally, but API response models hide it from public output.

PostgreSQL is better than JSON files for multi-user applications because it supports structured queries, persistent tables, safer concurrent access, indexing, and relationships between data. A JSON file is simple for learning, but it is not suitable when many users or requests need to read and update data at the same time.

Separating database models and Pydantic schemas improves security and maintainability. Database models can store internal fields such as image_path, created_at, updated_at, or password_hash, while public response schemas only expose safe fields such as id, name, price, email, and role. This prevents sensitive data from being accidentally returned to the frontend and keeps the API easier to change later.